

TVI-CAMS Regenerated Requirements Specification

Version: 1.1.0 Revision date: 2026-06-19 Status: Regenerated from current repository, legacy PDF, and Linear workspace Product name: TESDA Document and Compliance Manager / TVI-CAMS Prepared for: Product, Design, Engineering, QA, and Operations

1. Document Basis

This specification revises the legacy `training_compliance_system_requirements_3.pdf` into the current TVI-CAMS product direction. The legacy PDF is retained as an origin document, but its portfolio-demo assumptions are superseded by the current repository and Linear roadmap.

Reviewed Sources

Source	Treatment
<code>docs/training_compliance_system_requirements_3.pdf</code>	Legacy requirements v2.0. Retained for original Level 0-3 intent: batch visibility, urgency, NTP lag, documents, analytics, Supabase, Clerk, alerting, and PWA ideas.
<code>README.md</code>	Current MVP boundary and stack. Canonical for official TESDA systems boundary and no-Laravel MVP scope.
<code>docs/MASTER_PRD_SRS.md</code>	Prior consolidated source of truth. Reconciled with current repository and Linear updates dated 2026-06-17 to 2026-06-19.
<code>package.json</code>	Canonical installed stack: Next.js 16.2.6, React 19.2.4, TypeScript 5, Tailwind CSS v4, Clerk v7, Supabase JS v2.
<code>supabase/migrations/20260528160300_create_tenant_scoped_schema.sql</code>	Canonical implemented data model and RLS design.
Linear document TVI-CAMS Project Hub	Canonical roadmap, milestone sequence, role matrix, Figma screen index, schema inventory, and P0 demo gaps.
Linear issues TES-5 through TES-57	Canonical delivery backlog, priorities, statuses, acceptance criteria, and epic structure.
Current <code>app/</code> , <code>components/</code> , <code>lib/</code> code	Canonical implementation state and route/component inventory.

Precedence

When sources conflict, the order of authority is:

1. Current repository implementation and migration.
2. Linear Project Hub, active projects, and active issues.
3. Current repository documentation.
4. Legacy PDF requirements.

2. Executive Summary

TVI-CAMS is an internal compliance and document tracking system for Technical-Vocational Institutions and farm schools managing TESDA scholarship training batches, initially TWSP and CFSP.

The system tracks batch lifecycle status, document readiness, learner progress, LAMR evidence, trainer-side updates, audit activity, and internal billing preparation signals. It is not an official TESDA system and must not be represented as replacing TESDA SIS, T2MIS, BSRS, official approvals, official schedules, official reports, or final government records.

The product has evolved from the legacy portfolio concept into a multi-tenant, role-scoped operations tool using Clerk for authentication, Supabase Postgres/Storage/RLS for operational data, and a custom Next.js design system.

3. Current Tech Stack

Layer	Current technology
Framework	Next.js 16.2.6 App Router
UI runtime	React 19.2.4
Language	TypeScript 5
Styling	Tailwind CSS v4 with app/design-system.css and custom components
Icons	@tabler/icons-react plus local SVG assets
Authentication	Clerk @clerk/nextjs v7
Database	Supabase Postgres
Storage	Supabase Storage bucket compliance-evidence
Authorization	Supabase Row Level Security using Clerk JWT template named supabase
Auth to DB bridge	Clerk JWT passed as Authorization: Bearer; RLS reads auth.jwt() ->> 'sub'
Deployment target	Vercel
Design source	Figma custom design system and role-specific screens
Testing status	No real test runner yet; npm test is a no-op

Superseded Legacy Stack Assumptions

The legacy PDF referenced Next.js 14, shadcn/ui, Recharts, Resend, GitHub Actions cron, Upstash Redis, and PWA/offline mode. These are not current MVP dependencies. They may be considered future enhancements only when backed by Linear issues and implementation work.

4. Laravel API Integration Plan

Laravel may be introduced as an API-only backend and business-logic layer above Supabase. This is a future integration path, not a blocker for the MVP. The goal is to keep Next.js focused on presentation, move policy-heavy workflow logic into Laravel, and preserve Supabase as the operational database, storage system, and RLS safety layer.

Target Architecture

Next.js UI -> Laravel API -> Supabase Postgres and Supabase Storage

Retain:

- Next.js 16 and React 19 as the frontend.
- Clerk as the identity provider.
- Supabase Postgres as the operational database.
- Supabase Storage as the evidence file store.
- Supabase RLS as defense-in-depth for tenant isolation.

Add:

- Laravel 12 API-only application.
- Clerk JWT verification middleware.
- Laravel policies/gates for role, tenant, batch, document, trainer, and viewer access.
- API Resources or DTO transformers so the frontend never consumes raw database rows.
- Queue jobs for imports, exports, report generation, notifications, and long-running document workflows.

Placement

The Laravel API should live as a sibling application inside the repository unless deployment constraints require a separate repository:

```
tesda-compliance-manager-design-system-v0/  
app/ # Next.js frontend routes  
components/  
lib/  
supabase/  
api/ # Laravel API-only application
```

Phase 1: API Contract First

The existing API catalog in docs/MASTER_PRD_SRS.md should become the Laravel contract. Initial endpoints should include:

- GET /api/dashboard
- GET /api/batches
- POST /api/batches
- GET /api/batches/{batch}
- PATCH /api/batches/{batch}

- GET /api/documents/matrix
- POST /api/batches/{batch}/documents
- GET /api/trainer/classes
- POST /api/trainer/classes/{batch}/attendance
- GET /api/batches/{batch}/lamr
- POST/PATCH /api/batches/{batch}/lamr
- POST /api/import/batches
- GET/POST /api/export/batches
- GET /api/activity-log
- GET /api/reports
- GET/PATCH /api/settings/*

Acceptance criteria:

- Endpoint request and response DTOs are documented before implementation.
- Frontend receives user-friendly objects, not raw Supabase table rows.
- Error responses avoid SQL messages, table names, stack traces, and internal policy details.
- Route naming remains stable even if the backend changes from Next.js Route Handlers to Laravel.

Phase 2: Authentication and Authorization Bridge

Laravel must verify Clerk sessions directly and resolve application access before returning data.

Flow:

1. Next.js obtains the Clerk session.
2. Frontend calls Laravel with a Clerk JWT.
3. Laravel middleware verifies the JWT against Clerk JWKS.
4. Laravel loads profiles by clerk_user_id.
5. Laravel loads profile_tenant_memberships.
6. Laravel applies policies for tenant, role, batch, document, trainer assignment, and viewer read-only access.
7. Laravel returns role-scoped DTOs.

Rules:

- Tenant IDs from the browser are never trusted without server-side membership checks.
- Viewer write attempts are denied server-side.
- Trainer access is limited to assigned batches and allowed document audiences.
- Admin and Coordinator writes are limited to assigned tenants.
- Supabase RLS remains enabled and tested as a second enforcement layer.

Phase 3: Database and Storage Strategy

Laravel should connect to Supabase Postgres and Supabase Storage without replacing either service.

Preferred strategy:

- Laravel policies enforce application authorization.
- Supabase RLS remains enabled as defense-in-depth.
- Storage paths are built server-side by Laravel.
- File access uses short-lived signed URLs returned only after authorization.

Avoid:

- Using service-role credentials from frontend code.
- Bypassing tenant checks because Laravel is server-side.
- Returning database IDs or table names when user-facing identifiers are enough.
- Allowing arbitrary client-provided storage paths.

Recommended storage path pattern:

```
tenant/{tenant_id}/batch/{batch_id}/documents/{document_id}/{filename}
```

Phase 4: Laravel Module Structure

The Laravel API should separate controllers, policies, resources, services, and jobs.

```
api/app/Http/Middleware/VerifyClerkJwt.php
api/app/Http/Controllers/Api/DashboardController.php
api/app/Http/Controllers/Api/BatchController.php
api/app/Http/Controllers/Api/DocumentController.php
api/app/Http/Controllers/Api/TrainerController.php
api/app/Http/Controllers/Api/LamrController.php
api/app/Http/Controllers/Api/ImportController.php
api/app/Http/Controllers/Api/ExportController.php
api/app/Http/Resources/
api/app/Policies/
api/app/Services/
api/app/Jobs/
```

Business logic that should move behind Laravel services:

- Dashboard metrics.
- Urgency classification.
- Billing readiness.

- Document completeness.
- NTP-to-start lag.
- Trainer assignment checks.
- LAMR validation.
- Import validation and preview.
- Export generation.
- Activity logging.
- Signed evidence access.

Phase 5: Implementation Order

1. Scaffold a Laravel 12 API-only app under `api/`.
2. Add Clerk JWT verification middleware.
3. Add profile and tenant resolution service.
4. Add policies for Batch, Document, LAMR, TrainerClass, Settings, and Report access.
5. Implement read-only endpoints first: dashboard, batch list, batch detail, document matrix, trainer classes.
6. Switch Next.js data fetching from mock data to Laravel DTOs route by route.
7. Implement mutations: batch updates, document uploads, LAMR save, attendance/progress save.
8. Add import/export jobs and background processing.
9. Add activity logging to every mutation.
10. Add security, feature, and integration tests before real production data use.

Phase 6: Testing Gate

Laravel integration is not production-ready until automated tests prove:

- Cross-tenant reads and writes are denied.
- Viewer cannot create, update, delete, upload, import, or export restricted data.
- Trainer cannot open or mutate an unassigned batch.
- Trainer cannot upload admin/coordinator-only documents.
- Admin and Coordinator can manage only assigned tenant data.
- Every mutation writes an `activity_log` entry.
- Import rejects invalid files, malformed rows, duplicate batch IDs, and unauthorized tenant targets.
- Signed storage URLs are returned only to authorized users.
- DTOs omit restricted fields for Trainer and Viewer roles.

Linear Tracking

This integration should be tracked as a dedicated Linear issue or epic with the `api`, `security`, and `database` labels. It should be scheduled after the Supabase-only MVP backend contract is stable, unless the team decides Laravel is

required before live data onboarding.

5. Product Boundary

In Scope

- Clerk sign-in and protected routes.
- Role-scoped access for Admin, Coordinator, Trainer, and Viewer.
- Multi-tenant school scoping through Supabase RLS.
- Initial tenants: AKB Technical Vocational Inc., J3ED Farm School, and Nenita Farm Rice-Based School.
- Initial scholarship programs: TWSP and CFSP.
- Batch lifecycle tracking from AOU through completion or blocked state.
- Role-aware dashboards for Admin, Coordinator, Trainer, and Viewer.
- Document checklist and evidence management using configurable program requirements.
- Supabase Storage evidence upload and signed access.
- Trainer My Classes, attendance/progress updates, and trainer-scoped evidence upload.
- LAMR structured evidence: reports, outcomes, activities, entries, and source files.
- Internal billing preparation signal based on configurable program billing rules.
- Import/export workflows, including T2MIS/BSRS import mapping as a later milestone.
- Activity log and audit trail.
- Core analytics and reports.
- Settings, notifications, accessibility, RLS tests, and performance budgets as roadmap items.

Out of Scope for MVP

- Replacing TESDA SIS, T2MIS, BSRS, or official submission workflows.
- Official TESDA approvals or official billing approval.
- Direct official-system integration unless later authorized and technically feasible.
- Laravel backend requirements.
- Email notification delivery through Resend.
- GitHub Actions scheduled alerts.
- Upstash Redis caching or alert deduplication.
- MongoDB, OCR, AI document review, or document intelligence.
- PWA/offline operation.
- Public self-service registration.
- Super Admin role until schema, API, and audit requirements are approved.
- Requiring Laravel before the Supabase-backed MVP API contract is stable.

6. Users and Permissions

Action	Admin	Coordinator	Trainer	Viewer
Create/edit batches	Yes	Yes	No	No
View all tenant batches	Yes	Yes	Own assigned batches only	Read-only assigned tenant scope
Upload documents	Yes	Yes	Audience-scoped trainer/all documents only	No
Verify documents	Yes	Yes	No	No
Manage members and roles	Yes	No	No	No
Create or update LAMR entries	Yes	Yes	Own assigned batches only	No
View analytics and reports	Yes	Yes	No	Read-only
Import/export operational data	Yes	Yes	No	No unless explicitly granted later

7. Data Model Requirements

The implemented Supabase schema contains the following tables:

tenants, profiles, profile_tenant_memberships, scholarship_programs, program_document_requirements, program_billing_rules, batches, learners, documents, lamr_reports, lamr_outcomes, lamr_activities, lamr_entries, and activity_log.

Required Enums

Enum	Values
profile_role	admin, coordinator, trainer, viewer
lifecycle_stage	aou, ntp, tip, training, assessment, billing, completed, blocked
batch_status	pending, ongoing, completed, blocked
document_status	missing, pending, submitted, verified
document_audience	admin, coordinator, trainer, viewer, all
assessment_result	competent, not_yet_competent, pending
activity_action	created, updated, uploaded, verified, submitted, deleted, system_note

RLS Helper Requirements

Supabase policies must be backed by helper functions in app_private:

- `current_clerk_user_id()`

- `current_profile_id()`
- `current_role()`
- `can_access_tenant()`
- `can_manage_tenant()`
- `can_read_batch()`
- `can_trainer_write_batch()`

The Clerk JWT template must be named `supabase`. Both browser and server Supabase clients must pass the Clerk JWT so RLS can resolve the caller.

8. Functional Requirements

FR-01 Platform Foundation

The app must run as a Next.js 16 App Router application with Tailwind CSS v4 and the custom TVI-CAMS design system.

Acceptance criteria:

- The app has protected dashboard routes under `app/(dashboard)`.
- Shared shell components provide sidebar, topbar, mobile header, and metrics surfaces.
- Design components remain reusable and role-aware.
- Environment variables are documented for Clerk and Supabase.

Linear traceability: TES-5, TES-19, TES-30, TES-31.

FR-02 Authentication, Tenancy, and RBAC

Only authenticated users with active profiles and tenant memberships can access protected app data.

Acceptance criteria:

- Unauthenticated users are redirected to Clerk sign-in.
- Profiles support Admin, Coordinator, Trainer, and Viewer roles.
- Users may belong to one or more tenants.
- Every data read/write is scoped to tenant membership and role.
- Tenant/role resolution is available to every authenticated route and API handler.

Linear traceability: TES-6, TES-20, TES-34, TES-35, TES-36, TES-37.

FR-03 Role Dashboards

Admin, Coordinator, Viewer, and Trainer users need dashboard experiences aligned to their permissions.

Acceptance criteria:

- Admin and Coordinator dashboards show assigned tenant batches, metrics, lifecycle status, document readiness, and urgent work.
- Viewer dashboard hides all write controls and remains read-only in both UI and server/API behavior.
- Trainer dashboard shows assigned classes only.
- Role-specific dashboards support loading, empty, error, and permission-denied states.

Linear traceability: TES-8, TES-9, TES-12, TES-22, TES-40.

FR-04 Batch Lifecycle Tracking

The system must track every batch through the internal lifecycle:

aou -> ntp -> tip -> training -> assessment -> billing -> completed, with blocked allowed at any point.

Acceptance criteria:

- Batch cards and detail surfaces show current lifecycle stage, program, qualification, trainer, learner count, start/end dates, and current status.
- Lifecycle visuals distinguish done, active, pending, blocked, and completed states.
- The date-positioned Batch Timeline Gantt is tracked as a high-priority design-system rollout.
- Default operational sorting prioritizes urgent batches.

Linear traceability: TES-10, TES-56, TES-57.

FR-05 Deadline, Progress, and Urgency Engine

The system must compute progress, billing countdown, NTP lag, and urgency from source dates and program rules.

Acceptance criteria:

- Progress is computed as current training progress percentage and never displayed as an untraceable magic number.
- Billing urgency is classified as critical, warning, or on-track.
- NTP-to-start lag is visible and flagged when it exceeds the agreed threshold.
- Date math uses absolute source dates and a clear data-as-of timestamp.
- Impossible values are blocked or visibly flagged.

Linear traceability: TES-10, TES-14, TES-47.

FR-06 Documents and Evidence

The system must track required program documents, their statuses, and evidence files.

Acceptance criteria:

- Required documents come from `program_document_requirements`, not hardcoded UI branches.
- Users see human-readable document names, statuses, missing items, submitted files, and verification metadata.
- Evidence can be uploaded to Supabase Storage or linked through approved external URLs.
- File type and size validation happen before upload and server-side.
- Viewers can read scoped evidence but cannot mutate records.

Linear traceability: TES-11, TES-23, TES-43, TES-45.

FR-07 Trainer Workflows

Trainers need a focused workflow for assigned classes, attendance/progress, and trainer-side evidence.

Acceptance criteria:

- Trainers see only batches where they are assigned.
- Trainer views hide billing, BSRS, restricted documents, and tenant-wide compliance controls unless explicitly allowed.
- Trainers can mark attendance/progress and upload trainer/audience-all evidence for assigned batches.
- Trainer changes create activity log entries and respect RLS.

Linear traceability: TES-12, TES-24, TES-44, TES-45.

FR-08 LAMR Structured Evidence

LAMR must be represented as structured evidence plus source file references.

Acceptance criteria:

- LAMR captures TVI name, program title, batch section, module title, schedule, prepared by, approved by, and source file.
- Outcomes, activities, and learner entries are editable according to role permissions.
- LAMR APIs support CRUD for reports, outcomes, activities, and entries.
- LAMR data remains tenant-scoped and batch-scoped.

Linear traceability: TES-13, TES-25, TES-46.

FR-09 Internal Billing Preparation Signal

Billing preparation is an internal readiness signal, not official TESDA billing approval.

Acceptance criteria:

- Program-specific billing thresholds are read from `program_billing_rules`.
- Default threshold is 80 percent for TWSP and CFSP unless configured otherwise.

- Admin and Coordinator see billing preparation status.
- Trainer and Viewer do not see financial or restricted billing controls.
- The UI copy must avoid implying official billing approval.

Linear traceability: TES-14, TES-26, TES-52.

FR-10 Import and Export

The system must support tenant-scoped CSV import/export, with T2MIS/BSRS-specific mapping as a roadmap requirement.

Acceptance criteria:

- Standard CSV import validates file type, required headers, duplicate batch IDs, and row-level field errors.
- Valid imports preview before committing changes.
- Confirmed imports write only inside the selected tenant.
- Filtered exports include the current role/tenant-scoped result set.
- T2MIS/BSRS import overlay maps external fields into internal batch/learner fields.

Linear traceability: TES-15, TES-27, TES-49.

FR-11 Analytics and Reports

The system must provide operational analytics and report views for Admin, Coordinator, and Viewer roles.

Acceptance criteria:

- Analytics include training progress distribution, deadline countdown, document readiness, NTP lag, and active-batch alert counts.
- Charts include labels, units, empty states, and accessible summaries.
- Reports route decision must keep navigation free of broken links.
- Reports should cover enrolled, graduate, assessed, certified, and employed summaries where supported by available data.

Linear traceability: TES-26, TES-47, TES-48.

FR-12 Activity Log and Audit Trail

Every meaningful system action must be auditable.

Acceptance criteria:

- Batch, document, learner, LAMR, import, export, verification, deletion, and system note events are written to `activity_log`.
- Activity entries include tenant, actor, role, action, entity, timestamp, and relevant metadata.

- Viewers can inspect activity history in assigned scope.
- Activity log cannot expose cross-tenant or restricted information.

Linear traceability: TES-16, TES-27.

FR-13 Notifications

Notifications are a roadmap feature for surfacing operational alerts inside the app.

Acceptance criteria:

- Notifications drawer opens from the topbar bell.
- Drawer interactions are keyboard-accessible and focus-trapped.
- Unread counts are exposed through accessible labels.
- Alert preferences and auto-send controls require explicit policy before implementation.

Linear traceability: TES-50.

FR-14 Settings and Program Configuration

Admin users need configuration surfaces for members, roles, program documents, and billing rules.

Acceptance criteria:

- Admin can view tenant members and their roles.
- Role changes require confirmation or undo and create audit entries.
- Admin can view and edit program document requirements.
- Admin can adjust billing threshold values from 0 to 100.
- Changes affect future derived states without corrupting prior audit history.

Linear traceability: TES-51, TES-52.

9. Non-Functional Requirements

Security

- RLS must deny cross-tenant reads and writes for every tenant-scoped table.
- Storage access must prevent cross-tenant file reads and writes.
- Viewer write attempts must return server-side denial, not just hidden UI.
- Trainer access must be limited to assigned batches and allowed document audiences.
- Service-role usage must never be exposed to client code.

Linear traceability: TES-17, TES-37, TES-54.

Accessibility

- Modals and drawers must support keyboard-only operation.
- Dialog title and async status changes must be announced.
- Icon-only buttons need accessible names.
- Status must not rely on color alone.
- Core flows must satisfy WCAG 2.2 AA expectations.

Linear traceability: TES-53.

Performance and Reliability

- Dashboard LCP target: under 2 seconds on expected production data sizes.
- Common interactions target: under 300 ms perceived response where feasible.
- Mutation round-trip target: under 1 second for normal operations.
- Import preview target: under 5 seconds for supported file sizes.
- Server-side pagination or filtering is required once client-side tables exceed comfortable row counts.

Linear traceability: TES-55.

Testing

- Add a real test runner; `npm test` must stop being a no-op before production readiness.
- Cover coordinator batch compliance flow end-to-end.
- Cover trainer daily class flow end-to-end.
- Cover import preview and commit flow.
- Cover RLS/security with real Supabase policies, not mocked authorization.

Linear traceability: TES-18, TES-29, TES-54.

10. Delivery Roadmap

Milestone	Scope	Target
v0.1 Foundation	Auth, DB schema, RLS, middleware, dashboard shell	June 2026
v0.2 Core Batch Workflows	Batch CRUD, lifecycle, dashboards, urgency engine	July 2026
v0.3 Trainer and Documents	My Classes, document matrix, evidence upload, attendance	August 2026
v0.4 LAMR, Analytics and Billing	LAMR API, billing, analytics, reports, T2MIS import	September 2026
v0.5 Settings, Notifications and QA	Notifications, settings, RLS test suite, accessibility, performance	October 2026

Milestone	Scope	Target
Future Laravel API Layer	API-only Laravel service, Clerk JWT bridge, policies, DTOs, queues, imports/exports, report jobs	After MVP API contract stabilizes

11. Current Implementation Snapshot

As of 2026-06-19:

- Next.js app foundation, dashboard route group, auth pages, and shell components exist.
- Dashboard, analytics, documents, report, activity-log, trainer, trainer classes, attendance, and trainer document routes exist.
- Reusable UI components exist for batch cards, modals, status badges, urgency, progress, charts, empty states, and file preview.
- Supabase schema and RLS helper design exist in migration.
- Browser and server Supabase clients inject Clerk JWTs.
- Mock data facade still drives many UI surfaces.
- No app/api production route handlers were confirmed.
- No real automated test suite is configured.
- Storage path builder and signed access logic remain tracked work.

12. Priority Backlog Summary

Priority	Requirement
P0	Tenant/role middleware and route guards must be fully wired.
P0	RLS policies and helper functions must be verified with automated tests.
P0	Dashboard must use role-scoped live data or clearly isolated mock/demo mode.
P0	Supabase Storage path builder and signed evidence access must be implemented.
P1	Batch lifecycle, urgency, and document checklist must bind to live Supabase records.
P1	Trainer My Classes, attendance/progress, and document upload must respect assigned-batch scope.
P1	Activity logging must record all critical mutations.
P1	LAMR structured evidence module and API must be implemented.
P2	Analytics, reports, import/export, settings, and notifications must be completed according to roadmap.
P2	Accessibility and performance budgets must be tested before production sign-off.
P2	Laravel API integration must be scoped as a future business-logic layer above Supabase, not as a database replacement.

13. Legacy PDF Requirement Mapping

Legacy requirement	Current disposition
Display all batches	Retained, but scoped by tenant and role.
Training progress	Retained, computed from source data and visible in dashboards/analytics.
Billing deadlines	Retained as internal readiness countdown, not official approval.
NTP receipt and NTP-to-start lag	Retained.
TIP and assessment status	Retained.
BSRS exemption status	Retained for authorized roles only.
Linked documents	Replaced with configurable document requirements plus Supabase Storage evidence.
Batch lifecycle pipeline	Retained and expanded to include completed/blocked states and timeline/Gantt rollout.
Auto-generated remarks	Retained as derived status text where useful.
Summary metrics header	Retained through metrics components and dashboard cards.
Hardcoded document checklist	Superseded by program_document_requirements.
Static JSON start	Allowed only for demos; production must bind to Supabase.
Notion CSV migration	Not current source requirement; CSV import/export remains in scope.
Resend email alerts	Deferred.
GitHub Actions cron	Deferred.
Upstash Redis	Deferred.
PWA/offline	Deferred.
Clerk auth	Retained and implemented with Clerk v7.
Supabase live reads/writes	Retained and expanded with RLS and Clerk JWT bridge.

14. Acceptance Gate for MVP Demo

The MVP demo is acceptable only when:

- A user can sign in through Clerk and reach the correct role dashboard.
- Tenant and role are resolved for each protected route.
- Cross-tenant reads/writes are blocked by RLS.
- Admin/Coordinator can see batch lifecycle, urgency, document readiness, and billing preparation signals for assigned tenants.
- Trainer can see assigned classes and perform trainer-scoped updates only.
- Viewer can inspect assigned data without any write capability.
- Evidence upload paths are tenant-safe and signed access is enforced.
- Key mutations create activity log records.

- The UI communicates that TVI-CAMS is internal and not an official TESDA system.